AutoIA V1 User and Developer Manual

Target implementation: AutoIA `1.0.1`
Purpose: operational API reference, supported-language guide, source-code map, and developer extension manual.

This document is intentionally different from `AUTOIA_MATHEMATICAL_FOUNDATIONS_V1.tex`. The mathematical foundations document defines the proof objects and proves why they are sound. This manual explains how a user invokes those ideas and how a developer finds and changes their implementation without accidentally weakening the certification contract.

---

1. What AutoIA is

AutoIA certifies ranges of ordinary engineering Python functions over boxes of uncertain real inputs. A typical request has the form

```
from autoia import autoia, Interval


def model(p):
    x, y = p
    return x*x + 2.0*y

box = [Interval(0.9, 1.1), Interval(-0.2, 0.3)]
result = autoia(model, box)
print(result.lower_shaped)
print(result.upper_shaped)
print(result.machine_rigorous)
```

The returned interval is a proof result, not a sample-based estimate. If `machine_rigorous` is `True`, the accepted numerical proof obligations were evaluated by the audited binary64 trusted backend using outward-enclosing arithmetic.

The V1 contract is **fail closed**. If AutoIA does not understand an operation, cannot prove a required domain condition, exhausts a bounded path/subdivision budget, or encounters a numerical limit, it raises a typed `AutoIAError`. It does not silently return the nominal Python answer and label it rigorous.

---

2. Installation and version verification

Install a wheel in the Python environment used by your IDE or script. For example:

```
python -m pip install --upgrade --force-reinstall autoia-1.0.1-py3-none-any.whl
```

Then verify the interpreter and package version:

```
import sys
import autoia

print(sys.executable)
print(autoia.__version__)
```

Expected development-checkpoint version:

```
1.0.1
```

AutoIA does not require SciPy for its core arithmetic. SciPy is optional. If SciPy is present, AutoIA recognizes the supported SciPy semantics described later in this manual.

---

3. Public package surface

The top-level package exports the following groups.

3.1 Primary execution

```
from autoia import autoia, autoia_subdivide, autoia_adaptive_subdivide
```

- `autoia(model, parameter_intervals, ...)`: primary one-box entry point.
- `autoia_subdivide(model, parameter_intervals, split_counts=..., ...)`: user-prescribed box subdivision.
- `autoia_adaptive_subdivide(model, parameter_intervals, subdivision_policy=..., ...)`: structure-aware adaptive subdivision.

3.2 Repeated-query execution

```
from autoia import prepare
```

`prepare(model, ...)` constructs a persistent prepared session for repeatedly certifying the same engineering program on different boxes. It prepares the local semantic dispatch once, reuses one trusted backend, and may cache exact or restrictable certificates.

3.3 Input and policy objects

```
from autoia import (
    Interval,
    RangePolicy,
    SubdivisionPolicy,
    BranchPolicy,
    ConfigurationPolicy,
```

```
    RigorousBinary64Backend,
)
```

3.4 Result types

```
from autoia import (
    AutoIAResult,
    AutoIAConfiguredResult,
    AutoIAPathwiseResult,
    AdaptiveSubdivisionResult,
    PiecewiseResult,
)
```

The exact wrapper type depends on whether automatic configuration, uncertain control flow, and explicit subdivision were needed. The common user-facing fields are `lower`, `upper`, `lower_shaped`, `upper_shaped`, `width`, `width_shaped`, and `machine_rigorous`.

3.5 Public application-theorem interface

```
from autoia import (
    ApplicationBox,
    CertifiedInterval,
    CertificationContext,
    UpperBound,
    LowerBound,
    application_theorem,
    minimum_upper_bound,
    maximum_lower_bound,
)
```

This interface is for application packages that know an analytic theorem which can complement ordinary automatic enclosure. It intentionally exposes certified intervals and proof composition, not AutoIA's private polynomial or generator internals.

3.6 Public exception hierarchy

```
from autoia import (
    AutoIAError,
    InvalidInputError,
    UnsupportedOperationError,
    CertificationError,
    ResourceLimitError,
    NumericalLimitError,
    ModelExecutionError,
    InternalAutoIAError,
)
```

Catch `AutoIAError` for a general fail-closed boundary, or catch the specific subtype when the distinction matters.

---

4. The primary `autoia` call

The public signature is conceptually:

```
autoia(
    model,
    parameter_intervals,
    *,
    order=None,
    backend=None,
    representation=None,
    range_policy=None,
    range_method=None,
    internal_range_method=None,
    final_range_method=None,
    branch_policy=None,
    configuration_policy=None,
)
```

4.1 Novice mode

The intended V1 starting point is simply:

```
result = autoia(model, box)
```

When neither `order` nor `representation` is explicitly supplied, AutoIA uses automatic configuration. The release baseline is a polynomial representation of order 4. Bounded structural diagnostics may request a higher-order or shared-generator retry. Every successful attempt is independently rigorous; the returned numerical bounds are the componentwise intersection of successful certificates.

4.2 Expert/fixed mode

Supplying an explicit representation or order makes the request reproducible and expert-directed unless a configuration policy is also explicitly supplied:

```
result = autoia(
    model,
    box,
    representation="polynomial",
    order=6,
)
```

Supported V1 representations are:

- `"polynomial"`
- `"interval"`
- `"generator"`

Use fixed mode for controlled studies, debugging, benchmark comparisons, and reproducibility. Do not assume a higher order is always better; it can increase algebra and ranging cost.

---

5. Input boxes and endpoint semantics

Each uncertain scalar input is represented internally by a closed interval with finite binary64 proof endpoints. AutoIA 1.0.1 exposes three explicit public construction semantics.

5.1 Direct `Interval`: stored binary64 values are exact

```
box = [
    Interval(10.0, 12.0),
    Interval(0.2, 0.25),
]
```

This preserves the original V1 rule: after Python converts each argument to a stored binary64 value, AutoIA interprets that stored value as an exact real endpoint. The physical proof box is the Cartesian product

$$B = [\ell_1, u_1] \times \cdots \times [\ell_m, u_m],$$

where `m` is the number of uncertain inputs, `ell_i` is the stored lower endpoint, and `u_i` is the stored upper endpoint.

Point inputs whose stored binary64 value is itself the intended exact point are represented by equal endpoints:

```
Interval(3.0, 3.0)
```

5.2 Exact decimal/rational physical endpoints

A decimal source spelling such as `0.1` is converted to binary64 before an ordinary function call can inspect it. Therefore

```
Interval(0.1, 0.3)
```

does not mean the exact decimal interval `[1/10,3/10]`; it means the interval whose endpoints are the two stored binary64 values.

When the intended physical endpoints are exact decimals or rationals, use:

```
I = Interval.enclosing("0.1", "0.3")
```

Accepted exact endpoint types are decimal strings, `decimal.Decimal`, `fractions.Fraction`, and integers. For example:

```
from decimal import Decimal
from fractions import Fraction

A = Interval.enclosing("0.1", "0.3")
B = Interval.enclosing(Decimal("0.1"), Decimal("0.3"))
C = Interval.enclosing(Fraction(1, 10), Fraction(3, 10))
```

AutoIA converts the exact values to finite binary64 proof endpoints `L` and `U` and verifies, using exact rational comparison, that

$$L \le a \le b \le U,$$

where `[a,b]` is the requested exact-real interval. The ordinary float conversion is only a proposal; exact comparison establishes the containment. If the finite enclosure cannot be verified, the constructor fails closed.

`Interval.enclosing(0.1, 0.3)` intentionally raises `InvalidInputError`. Once `0.1` has become a float, AutoIA cannot recover whether the user meant the exact decimal `1/10` or the stored binary64 value.

A nonrepresentable exact point is also supported:

```
Interval.enclosing("0.1", "0.1")
```

It generally becomes a two-float proof interval bracketing the exact real point.

5.3 One-ulp padding of stored floats

```
I = Interval.outward(0.1, 0.3)
```

steps the supplied lower binary64 value once toward negative infinity and the upper value once toward positive infinity. This is explicitly a convenience pad. It proves that the original stored-float interval is contained in the padded interval, but it does **not** prove how an externally computed float was related to some unknown exact quantity before AutoIA was called.

5.4 Constants inside engineering code

The new input constructors do not reinterpret constants throughout user code. Ordinary constants retain the V1 semantics:

```
def model(p):
    return 0.37*p[0] + 12.5*p[1] - 4.8
```

The stored binary64 constants are exact real constants equal to their stored values. This is deliberate: a large existing engineering program should not require the user to mark every floating literal.

Use the following rule.

1. If a stored coefficient defines the numerical model, leave it unchanged.
2. If a coefficient is physically uncertain, make it an AutoIA uncertain input.
3. If a quantity is genuinely exact mathematically, prefer an exact formulation using exactly represented integers/rationals when practical or a supported semantic primitive. Do not assume that an ordinary literal `0.1` is silently reinterpreted as `1/10`.

5.5 Input provenance on results

Public one-box and prepared-session results expose:

```
result.input_semantics
result.input_specification
result.physical_input_box
```

For an `Interval.enclosing` input, `input_specification` records the requested exact endpoint forms, canonical exact rational values, stored proof endpoints, and `containment_verified=True`. For `Interval.outward`, it records the supplied stored floats and the padded proof endpoints. This metadata is descriptive; the numerical proof itself uses the stored proof interval.

For repeated-query APIs, explicit `Interval` objects preserve these semantics. Bare endpoint pairs or endpoint arrays continue to mean direct stored-binary64 endpoints, because no exact decimal/rational source information accompanies such arrays.

See `INPUT_SPECIFICATION.md` and Example 26 for the complete user decision rule.

---

6. Reading a result

For a scalar model:

```
r = autoia(model, box)
print(r.lower[0], r.upper[0])
```

For an array-valued model:

```
print(r.lower_shaped)
print(r.upper_shaped)
```

Useful common properties:

- `lower`, `upper`: flattened numeric lower/upper arrays.
- `lower_shaped`, `upper_shaped`: arrays reshaped to the model output shape.
- `width`, `width_shaped`: enclosure widths.
- `machine_rigorous`: whether the accepted proof used the rigorous backend.
- `representation_names`: representation description for structural results.
- `certificates`: proof/certificate records retained by the selected structural run.
- `input_semantics`: per-input constructor semantics used for the current request.
- `input_specification`: detailed requested/stored endpoint provenance records.
- `physical_input_box`: stored finite binary64 proof box actually certified.

An `AutoIAConfiguredResult` additionally provides:

- `attempt_results`
- `selected_result`
- `configuration_policy`
- `configuration_diagnostics`
- `configuration_summary`
- `explain_configuration()`

The configured numeric bounds can be tighter than the bounds in `selected_result` because the wrapper intersects several independently rigorous runs. The selected structural result remains available for inspecting path and representation metadata.

An `AutoIAPathwiseResult` additionally exposes:

- `branch_results`
- `branch_boxes`
- `branch_decisions`
- `branch_traces`
- `path_events`

These records explain which feasible execution paths were certified.

---

7. Supported scalar arithmetic

The normal arithmetic subset includes:

- addition and subtraction;
- multiplication;
- division when nonzero denominator can be proved, with bounded physical-domain refinement when necessary;
- negation;

- integer powers;
- selected rational/real power cases whose real-domain obligations can be proved;
- comparisons used by path-aware Python control flow.

The key user principle is that arithmetic is dependency-bearing. If the same uncertain variable appears twice, AutoIA does not intentionally replace those occurrences by unrelated scalar intervals at every operation.

Example:

```
def cancellation(p):
    x = p[0]
    return (x + 1.0) - x
```

A dependency-preserving representation can retain the common `x` structure and avoid the classical interval-dependency inflation that would arise from treating the two occurrences independently.

---

8. Elementary and statistical functions

The validated scalar semantic includes the central functions used throughout the examples and trusted layer, including:

- `sqrt`
- `exp`
- `log`
- `log10`
- `sin`
- `cos`
- `erf`
- standard-normal PDF/CDF helpers

AutoIA recognizes common NumPy/math spellings through its private local dispatcher. Real-domain requirements are proof obligations. For example, `sqrt(x)` requires a proof that the active physical path satisfies `x >= 0`; `log(x)` and `log10(x)` require `x > 0`.

If a broad enclosure obscures the domain even though smaller physical subboxes can prove it, the domain controller can request bounded physical-box refinement. AutoIA never fixes a failed proof by clipping an interval to the function domain.

---

9. Arrays and NumPy-style engineering code

AutoIA supports object-array based engineering assembly for the documented subset. Typical supported patterns include:

```python
import numpy as np


def model(p):
    x, y = p
    a = np.array([x, 2.0*y, x-y])
    return np.array([
        np.sum(a),
        np.dot(a, a),
    ])
```

The array layer and semantic dispatcher preserve AutoIA values through supported construction, broadcasting, reductions, and matrix composition.

Not every NumPy function is automatically certified. Unsupported functions must fail closed rather than coercing an AutoIA value to an ordinary float.

---

10. Linear systems

`np.linalg.solve` is recognized for the supported validated linear-system subset. AutoIA's semantic linear solve is not equivalent to running NumPy on interval objects. It uses nominal numerical linear algebra only as proposal/preconditioning information, then constructs a validated enclosure with the trusted arithmetic and dependency-bearing representation.

Example pattern:

```python
import numpy as np


def model(p):
    k = p[0]
    A = np.array([[2.0 + k, -1.0], [-1.0, 3.0]], dtype=object)
    b = np.array([1.0, 2.0], dtype=object)
    x = np.linalg.solve(A, b)
    return x
```

If the required nonsingularity/contraction proof cannot be established, certification fails closed.

---

11. Uncertain Python control flow

Ordinary branches such as

```python
def model(p):
    x = p[0]
```

```
    if x < 0.0:
        return x*x
    return x + 1.0
```

are handled by path-aware execution. If the input box crosses the branch surface, AutoIA certifies the feasible paths separately and returns the rigorous hull of their outputs.

11.1 Closed-boundary comparison semantics

V1 retains a proof-only exact affine shadow for expressions that remain exactly affine in the original physical inputs. This prevents harmless outward-rounding noise from changing feasibility at a closed endpoint.

For example, if `x` is physically restricted to `[0.95, 1.0]`, AutoIA can prove exactly that `x > 1.0` is impossible even if a floating-point range enclosure of `x - 1.0` would otherwise contain a tiny positive rounding extension.

The exact affine shadow is metadata for comparison proofs. It never replaces the outward-rounded numerical enclosure used for value certification.

11.2 Branch budgets

`BranchPolicy` controls bounded optional work:

```
from autoia import BranchPolicy

policy = BranchPolicy(
    max_branches=64,
    max_path_decisions=24,
    max_split_depth_per_decision=2,
    max_total_splits=64,
    max_domain_split_depth_per_obligation=6,
    prefer_geometric_split=True,
)
```

Resource budgets can make AutoIA stop earlier, but they cannot turn an invalid proof into a valid one.

---

12. Physical-domain refinement

A real-valued engineering model can be mathematically valid on a physical box while a broad dependency enclosure temporarily crosses a denominator zero or elementary-function boundary.

For example, if a denominator `d(x)` is truly nonzero on the physical box but a coarse enclosure contains zero, AutoIA can issue a domain-refinement request.

The controller subdivides the **physical input box**, re-executes the model independently on child boxes, and accepts only children on which the proof obligation succeeds.

This distinction matters:

- **proof refinement** subdivides because the current proof is too coarse;
- **domain clipping** would silently change the user's requested domain and is not allowed.

---

13. Range methods

A dependency-bearing enclosure still has to be ranged over its physical domain. `RangePolicy` controls these proof-preserving choices:

```
from autoia import RangePolicy

rp = RangePolicy(
    internal_method="termwise",
    final_method="adaptive",
    bernstein_max_coefficients=256,
    proof_bernstein_max_coefficients=256,
)
```

Available named methods are:

- `termwise`
- `monotonicity`
- `bernstein`
- `portfolio`
- `adaptive` for final ranging

The adaptive final policy can use cheap termwise bounds, derivative/monotonicity structure, reduced-dimensional Bernstein conversion, and subdivision subject to explicit costs. These are **tightening policies**. Soundness comes from the individual proof methods, not from the heuristic that decides which one to try.

---

14. Representations

14.1 Polynomial

The default representation retains a polynomial in centered physical variables plus a rigorous remainder. It is usually the best general first choice because it keeps dependence explicit and supports derivative/range analysis.

14.2 Interval

The interval representation is deliberately simpler. It can be useful as a robustness baseline or for operations where preserving higher-order structure is not useful.

14.3 Shared generator

The generator representation augments a retained polynomial with shared auxiliary uncertainty directions. It is useful when the same approximation error is reused and later recombined, because the shared generator can preserve that correlation rather than turning each occurrence into an independent interval remainder.

Use Example 19 to compare these modes explicitly.

---

15. Automatic configuration

`ConfigurationPolicy` is a scheduling policy for independent rigorous runs:

```
from autoia import ConfigurationPolicy

cp = ConfigurationPolicy(
    mode="auto",
    max_attempts=3,
    auto_order_increment=1,
    auto_try_higher_order=True,
    auto_try_generator=True,
)
```

Modes:

- `"fixed"`: run only the baseline.
- `"auto"`: baseline plus bounded structurally motivated retries.
- `"reference"`: explicit small research portfolio.

The release auto selector uses two general structural signals:

1. unstructured remainder dominates retained physical variation -> consider higher order;
2. approximation-error ancestry is reused/recombined -> consider shared generators.

A poor policy choice can waste work or miss a tighter result. It cannot make a successful independent certificate unsound. Successful configuration bounds are intersected componentwise.

---

16. Prescribed and adaptive subdivision

16.1 Prescribed subdivision

Use when a user wants a fixed partition for a convergence study or known physical decomposition:

```
r = autoia_subdivide(
    model,
    box,
    split_counts=[4, 8],
)
```

16.2 Adaptive subdivision

Use when the user explicitly wants a piecewise certificate and structure-aware refinement:

```
from autoia import autoia_adaptive_subdivide, SubdivisionPolicy

sp = SubdivisionPolicy(max_branches=32, max_depth=8)
r = autoia_adaptive_subdivide(model, box, subdivision_policy=sp)
```

Automatic domain/path refinement also occurs internally for specific proof obligations. That should not be confused with calling the public adaptive-subdivision API for output-tightening purposes.

---

17. Validated implicit nonlinear solves

17.1 `scipy.optimize.root_scalar`

For supported bracketed scalar roots, SciPy remains the nominal proposal engine. AutoIA independently verifies the mathematical root problem.

Typical code can remain ordinary SciPy:

```
from scipy.optimize import root_scalar


def residual(y, Re, eD):
    # engineering residual
    ...


def model(p):
    Re, eD = p
    sol = root_scalar(residual, bracket=[0.0008, 0.1], args=(Re, eD))
    return sol.root
```

V1 requires a finite constant bracket and supports the recognized bracketed methods. The proof establishes a unique root in the supplied bracket for every admitted physical input and returns a dependency-bearing implicit predictor plus rigorous correction.

17.2 `scipy.optimize.fsolve`

For a square nonlinear system, SciPy provides the nominal local solution. AutoIA constructs and verifies a local implicit solution branch using a validated contraction/Krawczyk-style tube.

The result proves uniqueness **inside the certified local tube**. It does not claim no other remote solutions exist.

17.3 Cross-module callbacks and helpers

Pure-Python helpers imported from another module are prepared **locally and lazily** when AutoIA semantic values reach them. The source module is not monkey-patched and ordinary numeric calls remain ordinary numeric calls.

Examples 15 and 16 demonstrate both same-module and cross-module implicit solves.

---

18. Fixed-data linear `RegularGridInterpolator`

V1 recognizes

`scipy.interpolate.RegularGridInterpolator`

for fixed finite numeric rectilinear grid data with `method="linear"` and uncertain query coordinates.

Example:

```
import numpy as np
from scipy.interpolate import RegularGridInterpolator

x_grid = np.array([0.0, 1.0, 2.0])
y_grid = np.array([0.0, 2.0, 5.0])
values = np.array([
    [0.0, 2.0, 5.0],
    [1.0, 3.0, 6.0],
    [4.0, 6.0, 9.0],
])
interp = RegularGridInterpolator((x_grid, y_grid), values, method="linear")
```

```
def model(p):
    return interp((p[0], p[1]))
```

Within one grid cell, AutoIA evaluates the exact tensor-product multilinear formula using dependency-bearing values. If an uncertain query crosses a grid boundary, the grid boundary becomes an ordinary path surface and the feasible cells are certified separately.

Supported V1 features include:

- ascending or descending grid axes;
- scalar or vector-valued fixed tables;
- cell-boundary crossing;
- `bounds_error=True`;
- finite constant `fill_value`;
- `fill_value=None` linear extrapolation;
- a fixed preconstructed interpolator imported from another source-backed Python module.

Not supported in this V1 semantic:

- uncertain grid coordinates;
- uncertain table values;
- interpolation methods other than `linear`;
- derivative `nu=` queries;
- general broadcasted tuple-query syntax beyond the documented subset.

See Examples 17 and 21.

---

19. Parameter-dependent SPD matrix families

The module `autoia/semantic/spd_matrix.py` implements application-independent validated semantics for symmetric positive-definite matrix families. The public surface includes:

```
from autoia import (
    certify_spd_matrix,
    logdet_spd,
    quad_solve_spd,
)
```

The NumPy dispatcher also recognizes SPD uses of:

```
np.linalg.slogdet(A)
np.linalg.det(A)
```

when `A` contains AutoIA values.

Proof architecture

For a dependency-bearing matrix family `A(p)`, the implementation:

1. proves structural symmetry;
2. constructs an ordinary floating-point lower-triangular proposal preconditioner `L`;
3. rigorously constructs `B(p) = L^{-1} A(p) L^{-T}` using `L` only as stored exact constants;
4. obtains independent symmetric eigenvalue enclosures from a norm theorem and interval Gershgorin bounds;
5. proves SPD if the rigorous eigenvalue lower bound is positive;
6. rescales `B = c(I+F)` so the symmetric matrix-log and inverse Neumann series have a rigorously bounded spectral radius below one;
7. retains matrix powers as ordinary AutoIA expressions and adds rigorous operator/trace tail bounds;
8. intersects these results with independent spectral bounds and, when available, the existing parameter-dependent linear-solve theorem.

The floating-point Cholesky factor is **proposal code**. It can make the theorem sharper or looser, but the proof does not trust it.

`SPDMatrixCertificate` contains diagnostics such as the proved eigenvalue interval, the proposal preconditioner, the series scale and radius, and the spectral log-determinant enclosure.

Developer rule

Do not convert a dependency-bearing matrix family to an entrywise independent interval matrix unless that loss of dependence is deliberate and documented. The whole purpose of `spd_matrix.py` is to reason about the actual common-parameter family rather than a fictitious Cartesian product of unrelated matrix entries.

---

20. Multicore CPU execution and optional GPU proposal helpers

Repeated independent boxes are handled by `autoia/session.py`. The public novice-facing aliases are:

```
from autoia import prepare, enclose_many, enclose_many_bounds
```

A prepared session supports:

```
results, info = session.enclose_many(
    boxes, workers="auto", parallel="auto", return_info=True
)
```

`parallel="auto"` resolves to serial execution for one worker, otherwise process execution when the model can be serialized, and threads as the fallback. Optional `cloudpickle` is preferred for spawn-process serialization when installed. In process mode every worker initializes one prepared AutoIA session and then certifies multiple boxes.

`ManyBoxExecutionInfo` is diagnostics-only metadata. It records the requested/resolved worker count and scheduling mode, chunk size, wall time, serializer, and trusted proof backend. No field in this object participates in a mathematical proof.

V1 execution-device policy

The preferred many-box calls, `enclose_many(...)` and `enclose_many_bounds(...)`, default to `workers="auto", parallel="auto"`. V1 uses a conservative scheduling heuristic: fewer than four chunks remain serial; otherwise the resolved worker count is capped by both the available CPU count and the number of chunks. Explicit user choices override this policy. `ManyBoxExecutionInfo.scheduling_note` records the resolved auto decision. Scheduling affects runtime only; every returned certificate is still produced by the same trusted CPU binary64 proof backend.

GPU proposal devices are different. `proposal_map(..., device="auto")` deliberately selects CPU in V1. MPS and CUDA must be requested explicitly because their performance advantage depends on the arithmetic intensity and transfer/dispatch cost of the caller's Torch function. MPS/CUDA remain non-rigorous proposal/assist paths and never become the proof backend through automatic selection.

Accelerator module

`autoia/accelerators.py` exposes:

```
available_accelerators()
proposal_map(...)
proposal_box_centers(...)
```

The V1 trust policy is strict:

- `cpu`: trusted for proof;
- `mps`: optional Apple Metal proposal/assist backend, never trusted for proof;
- `cuda`: optional NVIDIA proposal/assist backend, never trusted for proof.

`ProposalBatchResult.machine_rigorous` is always `False`. GPU values must be independently verified by ordinary CPU AutoIA certificates. The module intentionally requires a Torch-compatible batched proposal function; AutoIA does not attempt to translate arbitrary Python engineering code into Torch graphs.

`available_accelerators()` also reports diagnostic hardware information such as CUDA device name/compute capability when PyTorch exposes it. These fields are useful for portability testing but are not proof inputs.

---

21. Recognized unsupported `scipy.linalg.eigh`

AutoIA recognizes the common symmetric/generalized-symmetric eigensolver boundary:

```
scipy.linalg.eigh(A)
scipy.linalg.eigh(K, M)
```

but AutoIA V1 (including `1.0.1`) does **not** claim validated uncertain eigenvalue semantics. If an AutoIA-dependent matrix reaches `eigh`, the call raises `UnsupportedOperationError` with an explicit message stating that the operation is recognized but not implemented in this V1 build.

Numeric-only calls still delegate to ordinary SciPy.

This behavior is intentional. A future version can add theorem-backed eigenvalue certification without changing V1 into an accidental object-array failure or a false rigorous claim. See Example 18 and the V2 roadmap.

---

22. SciPy compatibility summary

Recognized V1-compatible SciPy semantics include the supported forms of:

- selected `scipy.special` functions such as `erf`/normal-CDF-related paths;
- common normal distribution PDF/CDF spellings;
- `scipy.optimize.root_scalar` under the validated bracketed contract;
- `scipy.optimize.fsolve` under the validated square-system local contract;
- fixed-data `scipy.interpolate.RegularGridInterpolator(..., method="linear")`;
- `scipy.linalg.eigh` as an explicit recognized **unsupported** boundary.

This is not a promise that arbitrary SciPy algorithms work with AutoIA values. Each supported algorithm needs a mathematically defined certification semantic.

---

23. Local semantic dispatch: what the code actually does

AutoIA does **not** monkey-patch NumPy, SciPy, Python builtins, or the user's imported modules globally.

Before validated execution, `autoia/semantic/dispatch.py` constructs a private shadow namespace for the source-backed user function. Supported NumPy/math/SciPy names are replaced only inside that prepared execution environment. Ordinary numeric execution outside AutoIA remains untouched.

When an imported source-backed pure-Python helper is reached with AutoIA semantic values, AutoIA can prepare that helper lazily. This is important for real engineering projects split across modules.

Important boundary: dynamic/local imports performed *inside* a function body are not equivalent to source-level module globals and remain a support boundary unless explicitly handled.

---

24. Prepared repeated-query sessions

For many boxes with one model:

```
from autoia import prepare, Interval

session = prepare(model)

r1 = session.certify([Interval(0.0, 1.0)])
r2 = session.certify([Interval(0.2, 0.8)])
print(session.stats())
```

Prepared execution separates:

1. static program preparation;
2. dynamic proof over each requested box.

The `CertificateCache` is a bounded LRU cache. It can answer:

- exact repeated-box queries;
- eligible interior-subbox queries by rigorously restricting a containing polynomial certificate.

Restriction is a proof transformation. It is not interpolation between cached answers.

Prepared many-box APIs also support serial/thread/process/chunked/streaming execution as documented by the session implementation and examples.

---

25. Application-theorem interface

Some application experts know a theorem that gives a rigorous bound more directly than generic automatic propagation. AutoIA allows those theorems to

coexist with automatic enclosure without putting application semantics into the core package.

A theorem receives a `CertificationContext`, which exposes proof-safe operations such as:

```
cert.point(...)
cert.interval(...)
cert.input_coordinate(...)
cert.enclose(...)
cert.intersect(...)
cert.sum(...)
cert.dot(...)
cert.square(...)
cert.maximum(...)
cert.exp(...)
cert.sqrt(...)
cert.log(...)
cert.log10(...)
cert.sin(...)
cert.cos(...)
cert.erf(...)
cert.standard_normal_pdf(...)
cert.standard_normal_cdf(...)
cert.solve_lower_triangular(...)
cert.upper_bound(...)
cert.lower_bound(...)
```

If two independent certified intervals enclose the same quantity, `cert.intersect` may rigorously tighten them. Independent upper bounds combine with a minimum; independent lower bounds combine with a maximum.

Application-specific GP/EI mathematics is an example of code that belongs in a package such as ExactBO, which depends on AutoIA rather than being embedded in AutoIA itself.

---

26. Typed failures and user response

`InvalidInputError`

The AutoIA call or policy is malformed. Fix the request.

`UnsupportedOperationError`

The model reached semantics that AutoIA has not certified. Rewrite using supported operations, use an application theorem, or treat the operation as an explicit product boundary.

`CertificationError`

The request is meaningful, but a proof obligation could not be established. Useful responses include tighter physical boxes, a physically justified bracket/initial guess, a stronger representation, or a larger but still finite proof-refinement budget.

`ResourceLimitError`

A finite path/subdivision/resource budget was exhausted. Increase it only when computationally justified.

`NumericalLimitError`

The trusted/reference arithmetic encountered overflow or another numerical limit.

`ModelExecutionError`

The user's ordinary model raised an exception unrelated to a validated semantic. Debug the model itself.

`InternalAutoIAError`

Treat this as an AutoIA bug and preserve a minimal reproducer.

A caught exception object provides `.code`, `.message`, `.context`, and `.suggestions`.

---

27. Recommended validation workflow for a new engineering code

1. Run the unchanged nominal model normally.
2. Wrap only the physical uncertain inputs into the `p` vector expected by AutoIA.
3. Start with `autoia(model, box)` and no expert knobs.
4. If it certifies, sample the original nominal model for a **diagnostic containment check**. Sampling is not the proof.
5. Perform a shrinking-box study. A useful enclosure should generally contract as the physical box contracts.
6. Inspect path/configuration diagnostics if the result is unexpectedly loose.
7. Only then try explicit representation, order, or subdivision settings.
8. Preserve any outside-code failure as a regression before modifying AutoIA.

For V1 engineering validation, classify every outside case as:

- certifies usefully;

- certifies rigorously but too loosely;
- fails closed with a good diagnostic;
- fails badly/ambiguously.

The last category is the priority product defect.

Part II. Code architecture for developers

28. Source tree map

The package is deliberately layered.

`autoia/api.py`

Public one-box/subdivision facade. Responsibilities include:

- public argument validation;
- novice versus expert configuration semantics;
- user-model exception guarding;
- translating internal exceptions into the stable public taxonomy;
- orchestrating automatic configuration and rigorous intersection of successful attempts.

This file should not become a home for new numerical proof algorithms.

`autoia/controller/`

Proof orchestration and optional work selection.

- `execution.py`: construct centered inputs, execute one structural certification, package `AutoIAResult`.
- `branching.py`: uncertain Python paths, geometric splits, path result aggregation, exact-affine branch surfaces.
- `configuration.py`: bounded automatic representation/order retries and intersection.
- `ranging.py`: structure/cost-aware final range portfolio.
- `subdivision.py`: prescribed/adaptive box subdivision.
- `split_scoring.py`: nontrusted heuristics for choosing promising splits.
- `policies.py`: internal policy dataclasses.
- `diagnostics.py`: controller/range diagnostic records.

Trust distinction: controller heuristics decide what sound proof attempt to perform. They do not define the truth of an enclosure.

`autoia/enclosures/`

Dependency-bearing value semantics.

- `value.py`: `ValidatedValue`, operator overloading, exact-affine metadata plumbing.
- `polynomial.py`: polynomial-plus-remainder enclosure operations.
- `interval.py`: interval enclosure representation.
- `generator.py`: polynomial plus shared-generator representation.
- `piecewise.py`: piecewise structural results.
- `elementary.py`: validated elementary propagation.
- `power.py`: power/reciprocal cases and domain obligations.
- `branching.py`: predicate objects/comparison semantics.
- `domain.py`: domain-proof/refinement request objects.
- `runtime.py`: active execution context.

`autoia/polynomial/`

Retained polynomial algebra and rigorous ranging support.

- `algebra.py`: multi-index polynomial operations.
- `packed.py`: batch-native packed coefficient operations.
- `ranging.py`: termwise, monotonicity, and Bernstein proof routines.
- `recenter.py`: rigorous polynomial certificate restriction/recentering.

`autoia/trusted/`

Small trusted numerical kernel. This is the most sensitive layer.

- `binary64.py`: directed/outward binary64 primitives.
- `interval.py`: trusted interval operations.
- `elementary.py`: trusted elementary interval enclosures.
- `reductions.py`: rigorous sums/dots/reductions.
- `array_ops.py`: trusted array-level operations.
- `environment.py`: runtime environment audit.

A change here requires substantially stronger review and high-precision testing than a controller heuristic change.

`autoia/semantic/dispatch.py`

Private local language bridge. It prepares a source-backed user function into an AutoIA-aware shadow namespace and supplies NumPy/math/helper proxies without global monkey-patching.

`autoia/semantic/scipy_dispatch.py`

SciPy recognition and dispatch. This is the public-language boundary for supported SciPy call forms.

Current important cases:

- normal/special APIs;

- `root_scalar`;
- `fsolve`;
- `RegularGridInterpolator`;
- explicit unsupported `eigh`.

`autoia/semantic/linear_solve.py`

Validated linear-system semantic.

`autoia/semantic/implicit_solve.py`

Validated scalar and square-system implicit solve semantics. Contains proposal-versus-proof separation, finite-difference/Jacobian preparation as applicable, and validated local/global proof obligations.

`autoia/semantic/interpolation.py`

Fixed-data multilinear `RegularGridInterpolator` semantic. Owns grid normalization, cell/path logic, query normalization, extrapolation/fill behavior, and dependency-preserving tensor-product evaluation.

`autoia/session.py`

Prepared/repeated-query API, certificate cache, certificate restriction, many-box execution, statistics.

`autoia/application/core.py`

Stable application theorem boundary. Keeps application-specific analytic theorems outside the AutoIA numerical core.

Public facade modules

- `intervals.py`
- `representations.py`
- `results.py`
- `policies.py`
- `backends.py`
- `special.py`
- `exceptions.py`

These stabilize names and public failure semantics even when internal research modules move.

---

29. End-to-end execution flow

For a normal `autoia(model, box)` call, the conceptual flow is:

```
public validation
    -> automatic/fixed configuration selection
        -> private semantic preparation of user model
            -> rigorous centering of physical input box
                -> construct dependency-bearing input values
                    -> execute engineering Python
                        -> supported semantic primitives
                        -> path/domain refinement if required
                    -> range each output rigorously
                -> structural certificate/result
        -> optional independent configuration retries
    -> componentwise intersection of successful certificates
-> public result
```

No controller stage is allowed to convert a failed proof into a successful one. Optional retries can only add independently certified information.

---

30. Representation internals developers must preserve

The conceptual invariant is

$$z(q) \in P(q) + G\epsilon + R.$$

Here:

- `q` is the centered physical input vector;
- `P(q)` is the retained multivariate polynomial;
- `epsilon` is a vector of auxiliary variables in `[-1,1]` used only by the generator representation;
- `G` contains their shared coefficients;
- `R` is an interval remainder.

An implementation change must preserve the enclosure for **every** physical point in the requested box and every admissible auxiliary generator value. Remainder terms may be widened but never silently dropped.

The exact affine comparison shadow is separate metadata. It is valid only while the expression is exactly affine in the original physical coordinates. Nonlinear operations must discard it unless an exact affine result is mathematically guaranteed.

---

31. Trusted versus untrusted/proposal code

A useful developer classification is:

Proof-critical trusted/reference arithmetic

If wrong, the returned certificate can be false.

Examples:

- outward rounding;
- interval arithmetic;
- trusted elementary enclosures;
- rigorous reduction error accounting;
- certified polynomial/remainder algebra;
- theorem verification inequalities.

Proposal code

May suggest centers, roots, Jacobians, preconditioners, cells, or candidate bounds. Its output must be checked independently before it affects the certificate.

Examples:

- SciPy `root_scalar` nominal root;
- SciPy `fsolve` nominal solution;
- NumPy/SciPy nominal linear algebra;
- future nominal eigenpairs.

Controller heuristics

Choose optional work. A bad choice can hurt speed or sharpness but must not hurt soundness.

Examples:

- split scoring;
- automatic higher-order retry trigger;
- generator retry trigger;
- range-method selection.

When reviewing a change, first identify which category it belongs to. Review and testing requirements should be proportional to proof criticality.

---

32. Adding a new elementary function

Do not start by adding a NumPy dispatch alias. The mathematical proof primitive comes first.

Required steps:

1. State the real domain precisely.
2. Define a trusted interval enclosure with outward rounding.
3. Define dependency-bearing polynomial/generator propagation and rigorous remainder treatment.
4. Add domain proof/refinement semantics if the real domain is restricted.
5. Add NumPy/math/SciPy spelling aliases only after the proof operation exists.
6. Test point values, broad boxes, boundary boxes, shrinking boxes, and high-precision containment.
7. Add at least one user-level example if the function is important enough to be a public feature.
8. Update the mathematical foundations and support matrix.

Never implement a function by evaluating the center and inflating by an empirical tolerance.

---

33. Adding a semantic primitive

A semantic primitive represents an algorithmic operation whose mathematical meaning is more useful than trying to run the original numerical algorithm on interval-like objects.

Examples already implemented:

- linear solve;
- bracketed scalar root;
- local square nonlinear solve;
- fixed-data multilinear interpolation.

Procedure:

1. Write the mathematical contract independently of the third-party implementation.
2. Separate **proposal** from **proof** explicitly.
3. Define what is globally certified and what is only local.
4. Decide whether the output can preserve dependence on original inputs.
5. Implement the proof in `autoia/semantic/` using trusted/enclosure operations.
6. Add local dispatcher recognition in `dispatch.py` or `scipy_dispatch.py` without global monkey-patching.
7. Ensure numeric-only calls still delegate unchanged.
8. Ensure unsupported variants fail with `UnsupportedOperationError`, not a coercion/type accident.
9. Add external-style regression cases, including imported-helper ownership when relevant.

10. Update docs and examples.

The nonlinear-solve implementation is the template for proposal-versus-proof algorithms. The interpolation implementation is the template for direct exact semantic replacement.

---

34. Adding a new SciPy operation

`autoia/semantic/scipy_dispatch.py` should remain a recognizer/adapter, not a dumping ground for proof mathematics.

For a new operation:

- preserve the real SciPy object/function for numeric-only calls;
- detect whether any semantically uncertain values are present;
- route uncertain calls into an AutoIA semantic module;
- preserve documented positional/keyword call forms deliberately;
- reject unsupported variants explicitly;
- support direct imported aliases when feasible;
- support module-qualified calls when feasible;
- test source-backed imported helpers.

If the mathematics is not implemented yet, an explicit recognized failure such as the V1 `eigh` boundary is preferable to a raw `object arrays are not supported` error.

---

35. Adding or changing a range method

A range method must prove that its returned interval contains the represented enclosure over the entire physical domain. It may use retained polynomial structure, monotonicity, Bernstein conversion, or another theorem.

A new method should provide:

- a clear proof statement;
- cost/size guardrails;
- a scalar/reference oracle when possible;
- comparison against high-precision or exhaustive small cases;
- monotonicity under box restriction where mathematically expected;
- controller integration that never makes the method mandatory for soundness unless it is itself fully trusted.

If a range method fails to tighten, the caller should retain an already rigorous enclosure rather than fail merely because an optional improvement was unavailable.

---

36. Adding or changing a representation

A new representation is a major architectural change. It must define:

1. concretization semantics: what set of real values the object represents;
2. construction of physical input variables;
3. addition/subtraction/multiplication/division/power semantics;
4. elementary-function semantics;
5. array/reduction compatibility;
6. range extraction;
7. path comparison semantics;
8. recenter/restriction behavior if prepared-cache restriction is claimed;
9. interaction with shared approximation-error ancestry;
10. conversion/intersection behavior with existing representations;
11. high-precision containment tests.

Do not expose a representation in `representation=` until these behaviors are coherent.

---

37. Changing the trusted kernel

Changes under `autoia/trusted/` require the strictest discipline.

At minimum:

- document the exact real operation being enclosed;
- identify all rounding points;
- ensure outward rounding on every bound-producing path;
- preserve finite/NaN/Inf failure semantics;
- audit reductions independently from scalar primitives;
- compare against high-precision arithmetic on adversarial inputs;
- run the complete regression suite and trusted-kernel audit;
- test on each supported architecture before release.

Performance optimization is never sufficient reason to replace a trusted path with an unchecked vectorized/library operation.

---

38. Testing map

Tests should be layered.

Unit/reference tests

Small algebra, trusted arithmetic, representations, range methods.

Semantic tests

NumPy/SciPy recognized calls, implicit solves, interpolation, branch behavior.

Failure tests

Unsupported operations, domain failures, resource limits, model exceptions.

Regression tests

Every externally discovered defect should become a permanent regression before the fix is declared complete.

High-precision audits

Use an independent arbitrary-precision implementation for representative point containment and theorem inequalities. The high-precision tool is an audit oracle, not runtime proof infrastructure.

Clean-wheel tests

Build the wheel, install it into an isolated target/environment, and run the canonical examples from the installed artifact rather than the source tree.

Application-boundary tests

Run dependent packages such as ExactBO against the installed wheel to catch accidental core/application coupling.

---

39. Canonical example suite

The V1 example suite is numbered `00` through `21`. The authoritative feature-to-example map is `examples/COVERAGE_MATRIX.md`.

The progression is:

- `00`: zero-knob smoke test
- `01`: scalar nonlinear arithmetic/functions
- `02`: arrays/reductions
- `03`: semantic linear solve
- `04`: uncertain branching
- `05`: subdivision
- `06`: prepared many-box execution
- `07`: shared-generator cancellation
- `08`: normal/special functions
- `09`: typed failure behavior
- `10`: EI-like generic nonlinear expression

- `11-14`: application theorem patterns
- `15`: validated `root_scalar` and `fsolve`
- `16`: cross-module implicit solves
- `17`: linear `RegularGridInterpolator`
- `18`: recognized unsupported `eigh`
- `19`: expert representations/range methods
- `20`: domain refinement and exact closed-boundary comparisons
- `21`: cross-module interpolation

Every release that changes semantics should run the full numbered suite from a clean installed wheel.

---

40. Debugging checklist

When a model unexpectedly fails:

1. Confirm the same function works numerically outside AutoIA.
2. Read the `AutoIAError` subtype and `.context`.
3. Identify the first unsupported/proof-sensitive operation, not merely the last stack frame.
4. Determine whether the issue is:
   - unsupported language semantics;
   - valid semantics but unresolved domain proof;
   - branch/path resource exhaustion;
   - enclosure blowup;
   - true model singularity/domain violation;
   - source dispatch boundary.
5. Reduce to the smallest ordinary-Python reproducer.
6. Add the reproducer as a failing test before modifying core code.
7. Prefer a general semantic/proof fix over an application-specific exception.
8. Rerun unrelated regression groups because dispatch/domain changes have broad reach.

When a result is rigorous but loose:

1. sample diagnostically;
2. shrink the box;
3. inspect path count;
4. compare fixed representations;
5. compare range methods;
6. try prescribed subdivision;
7. decide whether the issue is representation dependency loss, range looseness, or a controller choice.

Do not infer a soundness defect solely from a wide interval.

---

41. Release checklist

Before freezing a V1 release candidate:

- all tests pass in deterministic fresh-process groups;
- any monolithic-test-process instability is documented and investigated;
- all numbered examples pass from an isolated installed wheel;
- mathematical foundations compile;
- generated PDF is rendered and visually inspected;
- documentation support claims match code/tests;
- external engineering stress corpus is rerun;
- failure semantics are explicit for known unsupported boundaries;
- wheel contains only intended AutoIA package code;
- dependent application packages such as ExactBO pass against the wheel;
- wheel/source archive checksums are recorded;
- version numbers and documentation index agree.

A release should not be blocked merely because every possible engineering algorithm is unsupported. It **should** be blocked by incorrect containment, silent loss of rigor, misleading `machine_rigorous` status, broken packaging, or ambiguous behavior on operations claimed to be supported.

---

42. Architectural invariants to protect

The following are nonnegotiable unless a future major design review deliberately changes them:

1. **Fail closed.** No proof means no rigorous result.
2. **Outward-enclosing trusted arithmetic.** Floating-point implementation details cannot silently shrink a bound.
3. **Preserve dependence as long as practical.** Avoid premature independent intervalization.
4. **Proposal is not proof.** SciPy/NumPy algorithms may propose candidates but require independent validation.
5. **Controller heuristics are not trusted mathematics.** They schedule optional proof work only.
6. **No global monkey-patching.** Semantic preparation stays local to the certification execution.
7. **Application theorems stay outside AutoIA core.** AutoIA exposes a proof interface; packages such as ExactBO depend on it.
8. **External failures become general regressions.** Do not patch only the motivating application.
9. **Explicit boundaries are acceptable.** A precise `UnsupportedOperationError` is better than an unsound approximation.
10. **Documentation is part of the contract.** If the code and manual disagree, the release is not finished.

43. Where to read next

- `AUTOIA_MATHEMATICAL_FOUNDATIONS_V1.tex`: complete proof mathematics and notation.
- `AUTOIA_TUTORIAL_MANUAL_V1.md`: progressive runnable user tutorial.
- `../examples/COVERAGE_MATRIX.md`: capability-to-example map.
- `AUTOIA_V2_PATH_FORWARD.md`: proposed post-V1 research and engineering roadmap.
- repository-root design notes such as `IMPLICIT_SOLVE_SEMANTICS.md`, `PATH_AWARE_EXECUTION.md`, and the Step 20 verification records: implementation history and focused design rationale.

Appendix A. Exact public signatures at `1.0.1`

The following signatures were inspected directly from the checkpoint source. They are included here so this manual can serve as code documentation rather than only a conceptual guide.

Input interval constructors

```
Interval(lo, hi)
Interval.enclosing(lo_exact, hi_exact)
Interval.outward(lo_float, hi_float)
```

`Interval.enclosing` accepts exact decimal/rational specifications and rejects ordinary floats; `Interval.outward` performs one-adjacent-value padding of already stored floats.

```
autoia(
    model,
    parameter_intervals,
    *,
    order=None,
    backend=None,
    representation=None,
    range_policy=None,
    range_method=None,
    internal_range_method=None,
    final_range_method=None,
    branch_policy=None,
    configuration_policy=None,
)
```

Returns `AutoIAResult`, `AutoIAPathwiseResult`, or `AutoIAConfiguredResult` depending on the executed proof structure.

```
autoia_subdivide(
    model,
    parameter_intervals,
    split_counts,
    *,
    order=4,
    backend=None,
    representation="polynomial",
    range_policy=None,
    range_method=None,
    internal_range_method=None,
    final_range_method=None,
)
```

```
autoia_adaptive_subdivide(
    model,
    parameter_intervals,
    *,
    order=4,
    backend=None,
    representation="polynomial",
    range_policy=None,
    subdivision_policy=None,
)
```

```
prepare(model, **kwargs)
```

Returns `PreparedAutoIA` for a fixed configuration or `PreparedConfiguredAutoIA` for automatic/reference configuration.

Prepared fixed session

```
PreparedAutoIA(
    model,
    *,
    order=4,
    backend=None,
    representation="polynomial",
    range_policy=None,
    range_method=None,
    internal_range_method=None,
    final_range_method=None,
    branch_policy=None,
    cache_max_entries=256,
)
```

Important methods:

```
session.certify(parameter_intervals, *, use_cache=True, allow_restriction=True)

session.certify_many(
    boxes,
    *,
    workers=1,
    parallel="serial",
    chunk_size=64,
    use_cache=True,
    allow_restriction=True,
)

session.certify_many_bounds(boxes, **kwargs)
session.iter_certify_many(boxes, **kwargs)
session.iter_bounds_many(boxes, **kwargs)
session.clear_cache()
```

`PreparedConfiguredAutoIA` exposes the same repeated-query methods while managing the bounded configuration candidates internally.

Policy constructors

```
RangePolicy(
    internal_method="termwise",
    final_method="adaptive",
    bernstein_max_coefficients=256,
    proof_bernstein_max_coefficients=256,
)

SubdivisionPolicy(
    max_branches=16,
    max_depth=6,
    max_probe_directions=2,
    min_structural_gain=0.05,
    min_relative_width_gain=1.0e-3,
    split_method="cheap",
)

BranchPolicy(
    max_branches=64,
    max_path_decisions=24,
    max_split_depth_per_decision=2,
    max_total_splits=64,
    max_domain_split_depth_per_obligation=6,
    prefer_geometric_split=True,
)
```

```
ConfigurationPolicy(
    mode="auto",
    extra_orders=(5,),
    extra_representations=("generator",),
    max_attempts=3,
    auto_order_increment=1,
    auto_try_higher_order=True,
    auto_try_generator=True,
)
```

Application theorem constructors

```
application_theorem(
    *,
    name,
    target="quantity",
    theorem_status="user_asserted",
)

minimum_upper_bound(*bounds)
maximum_lower_bound(*bounds)
```

The theorem decorator does not prove an application theorem from scratch. It supplies the certified numerical context in which the user/application package evaluates a theorem whose assumptions it declares and checks.

Appendix B. V1 semantic support matrix

Operation family	V1 status	Important condition
Scalar `+ - * /`	Supported	Division needs a nonzero proof; may trigger physical refinement.
Constant powers	Supported subset	Real-domain rules depend on exponent/base.
`sqrt exp log log10 sin cos erf`	Supported	Restricted-domain functions require domain proof.
Normal PDF/CDF	Supported	Standard and documented constant-parameter forms.

Operation family	V1 status	Important condition
NumPy array assembly/reductions	Supported subset	See `ARRAY_SEMANTICS.md` and examples.
`np.linalg.solve`	Supported semantic	Parameter-dependent validated linear solve.
Python scalar `if/elif/else`	Supported subset	Path-aware finite branching under policy budgets.
`root_scalar`	Supported subset	Finite constant bracket; recognized bracketed methods; uniform proof.
`fsolve`	Supported subset	Square system; finite constant initial guess; local unique branch proof.
`RegularGridInterpolator` linear	Supported subset	Fixed finite grid/table data; uncertain query coordinates.
`scipy.linalg.eigh`	Recognized unsupported	Explicit fail-closed diagnostic for uncertain matrices.
`solve_ivp` / general ODE	Unsupported V1	V2 research target.
General optimization	Unsupported V1	Requires theorem-specific local/global semantics.
Arbitrary compiled-extension internals	Unsupported	No proof semantic merely from nominal execution.
Arbitrary dynamic local imports	Boundary	Source-backed prepared globals/helpers are the supported route.